# whitenoise — Workshop Guide

Stochastic White Noise Analysis with Memory

Version 0.2.0 · Python ≥ 3.9

## 1. Setup

```
# Google Colab
!pip install git+https://github.com/pnayga/whitenoise.git -q

# Local (from the package folder)
pip install -e .
```

```
import whitenoise as wn
import matplotlib.pyplot as plt
import numpy as np
```

## 2. CSV Format

```
time [months], sunspot_number [count]
1, 12.4
2, 15.1
```

- Row 1 = header (required)
- Column 1 = independent variable, Column 2 = observable
- Format: `name [unit]` — unitless: `flux []`
- Blank lines: `pd.read_csv(f, skip_blank_lines=True).dropna(how='all').to_csv('clean.csv', index=False)`

## 3. The Pipeline — Steps A to I

| Step | Function | What it does |
|---|---|---|
| A | `wn.read_csv(path)` | Load CSV → time, values, meta |
| B | `wn.plot_series(time, values, metadata=meta)` | Plot raw time series |
| C | `wn.compute_msd(values)` | Compute empirical MSD → lags, msd |
| D | `wn.plot_msd_empirical(lags, msd, metadata=meta)` | Inspect MSD shape |
| E | `wn.fit_msd(lags, msd, model='...')` | Fit model → FitResult |
| F | `print(fit.summary())` | Print parameters + 95% CI |
| G | matplotlib | Overlay fitted curve on MSD |
| H | matplotlib | Plot displacement PDF |
| I | `wn.analyze(path, model='...')` | Run A–H in one call |

```
# Steps A–D
time, values, meta = wn.read_csv('mydata.csv')
wn.plot_series(time, values, metadata=meta)
lags, msd = wn.compute_msd(values)
wn.plot_msd_empirical(lags, msd, metadata=meta)

# Steps E–F
fit = wn.fit_msd(lags, msd, model='cosine')
print(fit.summary())

# Steps G–H (manual plots)
fig, ax = plt.subplots()
ax.scatter(lags, msd, s=12, color='#555', alpha=0.6, label='Empirical')
ax.plot(fit.lags_used, fit.msd_fitted, color='#1B3A6B', lw=2, label='Fitted')
ax.legend(); plt.show()

T = int(lags[len(lags)//10])
displacements = values[T:] - values[:-T]
mu_disp = np.mean(displacements)
model_info = wn.get_model('cosine')
sigma2 = model_info['msd'](T, fit.params['mu'], fit.params['nu']) * fit.params['N']
dx = np.linspace(displacements.min(), displacements.max(), 300)
pdf = np.exp(-(dx-mu_disp)**2/(2*sigma2)) / np.sqrt(2*np.pi*sigma2)
plt.hist(displacements, bins='auto', density=True, alpha=0.5, color='#888')
plt.plot(dx, pdf, color='#C0392B', lw=2); plt.show()

# Step I — equivalent one-liner
result = wn.analyze('mydata.csv', model='cosine')
result.summary()
wn.plot_diagnostics(result)
```

## 4. Memory Parameter μ

| μ | Regime |
|---|---|
| < 0.95 | Subdiffusive |

| | |
|---|---|
| 0.95–1.05 | Near-Brownian |
| 1.05–2.0 | Superdiffusive |
| > 2.0 | Hyperballistic |

For fBm: H < 0.5 subdiffusive · H = 0.5 Brownian · H > 0.5 superdiffusive

## 5. Available Models

| Model | Parameters | MSD formula | Typical use |
|---|---|---|---|
| `cosine` | μ, ν | $\sqrt{\pi}\,\Gamma(\mu)\cos(\nu T/2)\,J_{\mu-1/2}(\nu T/2)\,(T/\nu)^{\mu-1/2}$ | $CO_2$, X-ray, coral |
| `exponential` | μ, β | $\Gamma(\mu)\,\beta^{-\mu}\,T^{\mu-1}\,e^{-\beta/T}$ | Sunspots, earthquakes |
| `sine` | μ, ν | Same as cosine with sin(νT/2) | Population dynamics |
| `fbm` | H | $T^{2H}$ | Self-similar processes |
| `dna` | a, b, c | $a - c\cdot e^{-bL}$ (plateau) | Nucleotide separations |

$R^2$ guide: ≥ 0.8 good · 0.5–0.8 moderate · < 0.5 try another model

```
wn.list_models()                                  # see all models
result = wn.analyze('mydata.csv', model='cosine')  # try one
cr = wn.batch_model_search('mydata.csv')           # try all, compare R²
wn.print_comparison(cr)
```

## 6. Demo Datasets

| File | System | Model | Expected μ |
|---|---|---|---|
| `exponential_sunspot.csv` | Solar sunspot | exponential | ~1.12 |
| `cosine_co2.csv` | $CO_2$ Keeling | cosine | ~0.93 |
| `cosine_xray.csv` | X-ray binary | cosine | ~1.22 |
| `sample_series.csv` | Synthetic fBm | fbm | H ≈ 0.72 |
| `dna_nucleotide.csv` | Bacterial genome | dna | a≈5.2, b≈0.002 |

### Colab — download datasets:

```
base = 'https://raw.githubusercontent.com/pnayga/whitenoise/main/sample_data/'
for f in ['exponential_sunspot.csv','cosine_co2.csv','cosine_xray.csv',
          'sample_series.csv','dna_nucleotide.csv']:
    !wget -q {base}{f}
```

### Local — files are in `sample_data/`:

```
time, values, meta = wn.read_csv('sample_data/exponential_sunspot.csv')
```

## 7. Preprocessing (optional)

| Function | Options | Use when |
|---|---|---|
| `wn.detrend(values, method='linear')` | 'linear', 'polynomial', 'mean', 'moving_average' | Data has a trend |
| `wn.normalize(values, method='zscore')` | 'zscore', 'minmax', 'mean', 'maxabs' | Data amplitude is very small or large |
| `wn.smooth(values, window=5)` | 'moving_average', 'gaussian' | Noisy signal |

```
fluct = wn.detrend(values, method='linear')
wn.plot_series(time, fluct, title='Detrended')

# Compare before/after
fig, axes = plt.subplots(2, 1, figsize=(10, 5), sharex=True)
wn.plot_series(time, values, metadata=meta, title='Original',  ax=axes[0], show=False)
wn.plot_series(time, fluct,  metadata=meta, title='Detrended', ax=axes[1], show=False)
plt.tight_layout(); plt.show()
```

**Note:** The `N` parameter in the fit ( `N · msd_model` ) absorbs data amplitude — μ is scale-independent. Normalize only if `N` collapses to ~0 in the fit output.

## 8. Comparing Multiple Datasets

```
cr = wn.compare({
    'System A': 'a.csv',
    'System B': 'b.csv',
}, model='cosine')
wn.print_comparison(cr)
wn.publish_comparison(cr)
```

## 9. Quick Reference

| Function | Returns |
|---|---|
| `wn.read_csv(path)` | time, values, meta |
| `wn.read_csv_multi(path)` | list of (time, values, meta) |
| `wn.compute_msd(values)` | lags, msd |
| `wn.fit_msd(lags, msd, model)` | FitResult or None |
| `wn.analyze(path, model)` | AnalysisResult |
| `wn.compare(datasets, model)` | ComparisonResult |
| `wn.batch_analyze(source, model)` | ComparisonResult |
| `wn.batch_model_search(path)` | ComparisonResult |
| `wn.plot_series(x, values, metadata=meta)` | Figure |
| `wn.plot_msd_empirical(lags, msd, metadata=meta)` | Figure |
| `wn.plot_msd(result)` | Figure |
| `wn.plot_pdf(result, lag_index=None)` | Figure |
| `wn.plot_diagnostics(result)` | Figure (2×2) |
| `wn.publish_msd(result, save_path=None)` | Figure |
| `wn.publish_comparison(cr, save_path=None)` | Figure |
| `wn.export_csv(result, path)` | saves CSV |
| `wn.detrend(values, method)` | ndarray |
| `wn.normalize(values, method)` | ndarray |
| `wn.smooth(values, window)` | ndarray |
| `wn.list_models()` | prints table |

| | |
|---|---|
| `wn.get_model(name)` | model info dict |